

C语言与程序设计 The C and Programming

第3章 基本的标准输入与输出

王多强

1097412466

dqwang@hust.edu.cn

华中科技大学计算机学院

输入：将计算机外部的数据输送到计算机内部；

输出：将计算机内部的数据输出到计算机外部。

- ◆ **输入、输出是计算机与外界交互的手段**

- ◆ 计算机有两种基本的输入输出类型：**流式数据**和**格式化数据**

- **流式数据**：不区分数据的类型，没有输出样式的要求，仅将数据视为一个一个字节组成的“流”（称为**字节流**）。

- **格式化数据**：区分数据类型，并有输入输出样式的要求。

- ◆ 本章介绍一些C语言的标准输入输出函数的使用。

- ◆ C语言提供了一组**函数**来实现输入输出操作

常用的标准输入、输出函数：

1. **单个字符**输入、输出函数： `getchar`、`putchar`
2. **字符串**输入、输出函数： `gets`、`puts`
3. **格式化**输入、输出函数： `scanf`、`printf`

注：

标准输入： 从键盘输入

标准输出： 向屏幕输出

非标准输入输出：

- ◆ **从一般文件输入**
- ◆ **向一般文件输出**

3.1 字符的输入输出

1、单个字符的输入函数getchar

函数原型: **int getchar();**

功 能: 接收从键盘输入的一个字符, 返回接收到的字符的
ASCII码 (整型) 。

基本使用方法:

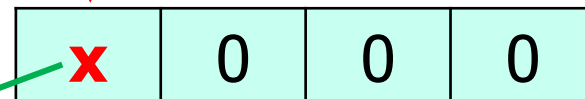
设: **char c;**

c = getchar();

参数表中没有参数

最低字节

小端表示



输入的字符
的ASCII码

int型返回值, 最低字节
的内容赋给x:

c ← x

2、单个字符的输出函数putchar

函数原型: **int putchar(int ch);**

功 能: 将参数表中的字符输出至屏幕 (以"图形方式"显示)。

基本使用方法:

设: **char c='A';**

putchar(c);

↑
一般不用
管返回值

返回值: 所输出字符的ASCII码 (int整数); 执行异常则返回**EOF**。

注： putchar函数的参数原型是**int型**，使用时有多种形式：

- ◆ **字符变量**作为参数：`char c = 'a'; putchar(c);`
- ◆ **字符常量**作为参数：`putchar('a');`
- ◆ **转义字符**作为参数：`putchar('\141');`
- ◆ **小整数(ASCII码)**作为参数：`putchar(97);`
- ◆ **其它int型整数**作为参数：`int i = 0x80802061;`
`putchar(i);` 或
`putchar(0x80702061);`

整
数
提
升

- ◆ 当参数的数据长度大于1个字节时，仅输出**最低字节**的内容。

0x61

0x20

0x70

0x80

例3.2 使用putchar函数打印 “HUST” 字样。

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
    char c1, c2;
```

```
    short c3;
```

```
    int c4;
```

```
    c1 = 'H'; c2 = 'U'; c3 = 83; c4 = 'T';
```

```
    putchar(c1); putchar(c2); putchar(c3); putchar(c4);
```

```
    return 0;
```

```
}
```

例3.1 getchar函数的调用。

```
#include <stdio.h>
```

```
int main(void)
```

```
{ char ch1, ch2, ch3;
```

```
  ch1=getchar(); /* 读到字符a */
```

```
  ch2=getchar(); /* 读到换行符\n */
```

```
  ch3=getchar(); /* 读到换行符b */
```

```
  printf("\n%c%c%c", ch1, ch2, ch3);
```

```
  printf("\n%d %d %d", ch1, ch2, ch3);
```

```
  return 0;
```

```
}
```

第二个getchar实际
读入的是换行符(10)

设输入：

a ✓

b ✓

这里输入了一个换行符

则输出：

a ✓

b

这里输出了读入的换行符

97 10 98

ch2的值是10

分析：操作系统中有一个称为“**输入流**”的内存缓冲区。

“敲键盘” 输入的字符首先存储在输入流中。而输入函数(如 getchar、scanf等) 将从输入流中的读取数据（字符）。

如：假定输入：abc ✓

输入流缓冲区的内容

a	b	c	\n		
---	---	---	----	--	--

换行符也进入输入流

getchar函数的执行：

getchar函数执行时，先检查**输入流**中是否有字符：**若有**，则读出输入流中的第一个字符，然后返回其值；若无，则使程序进入等待状态，而当有了输入，并**按下回车键**后，激活处于等待状态的getchar函数，然后getchar函数继续执行（继续从输入流中获取输入的字符）。

输入： 输入流：

a ✓ →

a	\n				
---	----	--	--	--	--

\n					
----	--	--	--	--	--

`ch1=getchar();`

a 赋给ch1

`ch2=getchar();`

\n 赋给ch2

b ✓ →

b	\n				
---	----	--	--	--	--

\n					
----	--	--	--	--	--

`ch3=getchar();`

b 赋给ch3



最后一个\n留在输入流中

注：字符一旦被读出，输入流中
就将其清除，以便读后续字符。

如何消除换行符的影响：多调用一次getchar，把不必要的换行符读出来。

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
    char ch1, ch2, ch3;
```

```
    ch1=getchar(); /* 读到字符a */
```

```
    getchar();    “空读”一下，清掉换行符
```

```
    ch2=getchar(); /* 读到字符b */
```

```
    ch3=getchar(); /* 读到字符\n */
```

```
    printf("\n%c%c%c", ch1, ch2, ch3);
```

```
    printf("\n%d %d %d", ch1, ch2, ch3);
```

```
    return 0;
```

```
}
```

设输入：

a ✓

b ✓

则输出：

ab ✓ 这里输出换行符

97 98 10

3.2 字符串的输入与输出

1、字符串输出函数puts

函数原型: `int puts(char *str);`

参数为**字符指针**,
这里先理解为字符串

功 能: 将字符串str的内容输出到屏幕上, 并**自动换行**;

返 回 值: 输出的字符数, 如果出错则返回EOF。

基本调用形式:

设有: `char s[5] = "abc";`

`puts(s);`

一般不用管返回值

或 `puts("abc");`

例 使用puts函数在第一行输出"Hello,"，在第二行输出"World!"。

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

s是字符数组，里面
存着字符串 "Hello,"

```
char s[20] = "Hello,";
```

```
char * pc = "World!";
```

```
puts(s);
```

pc是**字符指针**，指向字符串常量 "World!"

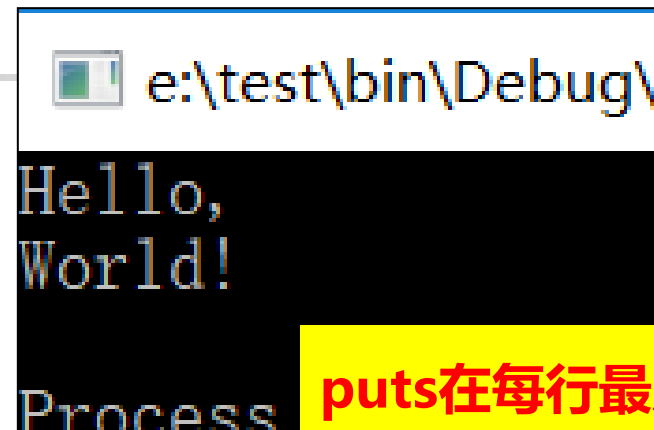
```
puts(pc);
```

使用时，用字符指针pc引用字符串即可。

```
return 0;
```

```
}
```

输出：



```
e:\test\bin\Debug\  
Hello,  
World!  
Process
```

puts在每行最后
自动加换行符。

2、字符串输入函数gets

函数原型: `char * gets(char * str);`

参数也为**字符指针**,
这里先理解为**字符数组**

功 能: 接收从键盘输入的**以回车键结束**的一行字符, 作为字符串存入**字符数组str**中。

返 回 值: str的地址 (即字符指针str的值) 。

基本调用形式: 设有: `char str[5];` 定义字符数组

`gets(str);`

一般不用管返回值

第一章就用过:

```
printf("Input your name please!\n");  
gets(name);
```

使用gets存在缓冲区溢出风险，要注意“安全”：

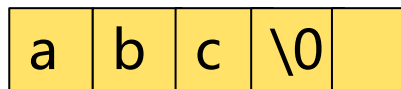
将用于存储输入数据的内存空间称为“**接收缓冲区**”（如gets函数中使用的字符数组，同理，输出时有“**输出缓冲区**”）。

gets的执行：从当前输入流中的读取字符序列，直到碰到换行符\n为止，然后将 \n 替换成**字符串结束标志\0**；这一过程中读取到的所有字符（包括\0）存入参数指定的**接收缓冲区**。

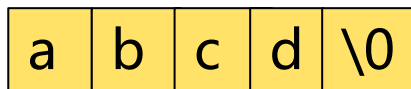
因此**要求接收缓冲区的长度（字节数）至少不能小于输入的字符数+1**。但**gets不检查接收缓冲区的长度**。而是输入多少字符就存多少字符。如果接收缓冲区的长度小于**实际输入的字符数+1**，则会“存储溢出”而造成严重问题。

如: `char s[5]; gets(s);`

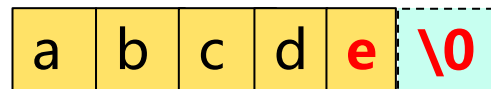
输入"abc", 正常



输入"abcd", 正常



输入"abcde", 溢出



因此, 使用gets的时候, 程序员要清楚缓冲区的大小, 避免发生溢出风险。

◆ 安全的字符串输入函数: `gets_s`:

`gets_s`在参数里指定缓冲区大小, 并**主动检查缓冲区大小**, 最多只接受指定数量的字符。

如: `gets_s(s, 5);`

3.3 格式化输入与输出

格式化输入输出：按照指定的样式输入、输出。

1. 格式化输出函数printf

函数原型: `int printf(const char *format, .....);`

格式字符串 其它参数

功 能：按照**格式字符串**规定的格式，将格式串内容输出到屏幕上，返回值是实际输出的字符个数。

基本调用形式:

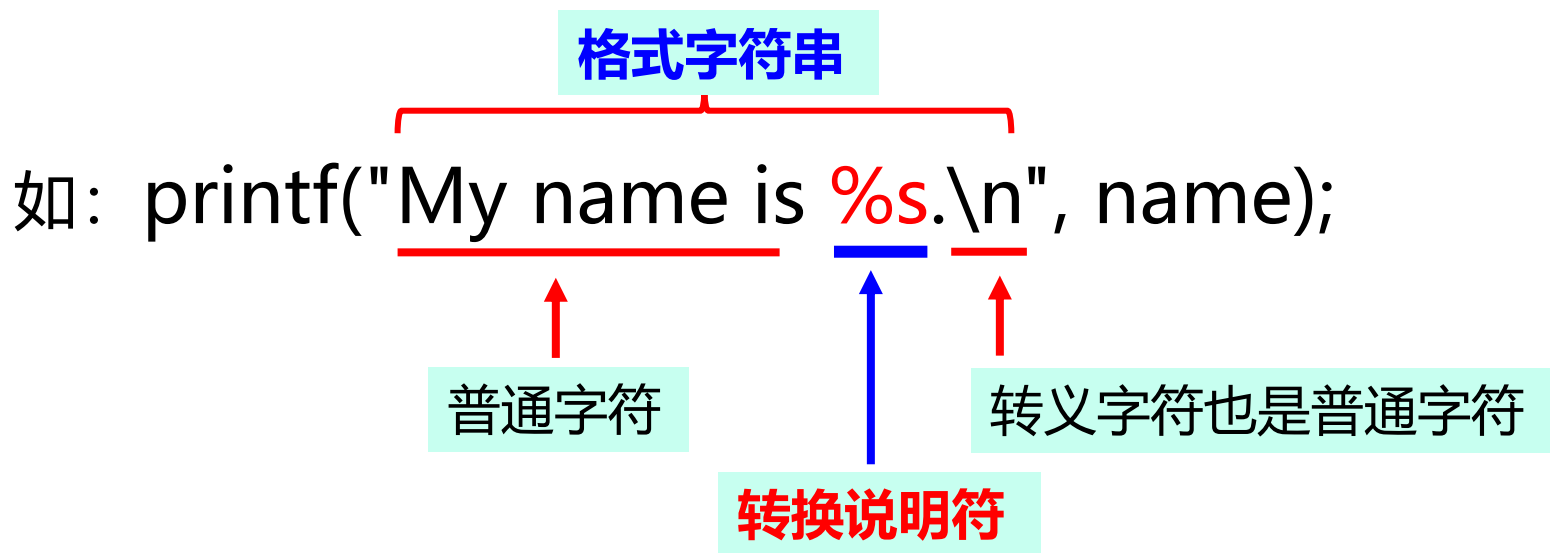
设有: `char name[10] = "Wang";` 转换说明字符

`printf("My name is %s.\n", name);`

格式字符串

17

格式字符串：是由**普通字符**和**转换说明字符**组成的字符串。



- ◆ **转换说明字符**是以**%**开始的特殊字符序列，起到**占位符**的作用。
- ◆ 输出时，**普通字符原样输出**，**转换说明字符**不直接输出，而是用后面的参数的值对应代换后输出。

设name的内容是："Wang"，则输出为：**My name is Wang.**

printf的调用

必须有

printf(**格式字符串**, **数据项1**, ..., **数据项n**);

- ◆ 如果格式字符串中有转换说明，就必须有对应的数据项，
- ◆ 否则不用写数据项。

特别注意：数据项的数目、类型、顺序都必须与格式字符串中的转换说明一一对应。否则，编译时通常不会报错，但是执行时不能得到正确的输出结果。

(1) 转换说明字符的语法形式: %**[域宽说明]****转换字符**

- ◆ **%**: 转换说明的前导符, 必须有。
- ◆ **域宽说明**: 用来指出数据输出时的格式要求。如**对齐方向**、**输出数据域的宽度**、**小数部分的位数**等。
- ◆ **转换字符**: 用来指出所要输出的**数据的类型**。

重点掌握以下4个格式转换符

- ◆ **d: %d**, 输出整数
- ◆ **c: %c**, 输出字符
- ◆ **s: %s**, 输出字符串
- ◆ **f: %f**, 输出浮点数

例：设变量说明为：

```
char c=65;                int x1=65;
unsigned x2=65535;         float y=65;
char name[100]= "The C programming" ;
```

则

7个格式转换说明 7个对应的数据项参数

```
printf("%c,%d,%d,%c,%u,%f,%s",c, c, x1, x1, x2, y, name);
```

输出： A,65,65,A,65535,65.000000,The C programming

默认情况下，浮点数输出精度为小数点后6位。

(2) 域宽说明

%[域宽说明]转换字符

域宽说明用来指出输出数据的**对齐方式**、**输出数据域的宽度**、**小数部分的位数**等要求。

常用的域宽说明字符：

- ：输出时，数据左对齐。没有时默认为右对齐；
- m**：m是个整数，给出输出的最小域宽，即至少输出m个字符的宽度。实际数据不够时，需要补空格；
- .n**：n是个整数，对整数、浮点数、字符串有不同的含义；
- h**：输出短整数；
- l**：输出长整数；
- L**：输出长双精度浮点数。

例 输出整数：-和m.n的用法

.n：最少要输出n个数字，不足时，前面补0。

设： int i1=12;

```
printf( "i1=%-8.3d,i1=%8.3d\n" , i1, i1);
```

输出： i1=012 ,i1= 012

左对齐，8位域宽（最少输出8个字符宽度），不足时后面补空格；最少3个数码，不足时前面补0

右对齐，8位域宽（最少输出8个字符宽度），不足时前面补空格；最少3个数码，不足时前面补0

当实际数据长度大于m或n时，仅原样输出，不受m或n的限制。

如： int i2=12345;

```
printf( "i2=%-8.3d,i2=%8.3d\n" , i2, i2);
```

输出： i2=12345 ,i2= 12345

输出浮点数：-和m.n的用法

.n：小数点后的位数，不足时补0，多时四舍五入。

设： double y=123.4567;

```
printf("y=%4.2f\n", y);
```

输出： y= 123.46

默认右对齐，4位域宽（总体最少输出4个字符宽度，**包括小数点**），超出时原样输出。
小数点后2位精度，多的四舍五入。

```
float x=3.1416;
```

```
printf("x=%8.3f, x=%-.6f\n", x, x);
```

输出： x= 3.142, x= 3.141600

默认右对齐，最小8位域宽，不足时前面补空格。小数点后3位。多的四舍五入

左对齐，没有域宽要求，原样输出。小数点后6位，不足时补0

输出字符串时，-和m.n的用法

.n：只输出n个字符，多的不输出，少的补空格。

```
char s1[20]="Technology";
```

```
printf("%8s,%-8.4s,%8.4s\n", s1, s1);
```

输出： Technology, Tech , Tech

默认右对齐，
最少输出8个
字符宽度，不
足时前面补空
格，超出的原
样输出

左对齐，8位
域宽，但只输
出4个字符，
少的部分补后
面空格。

右对齐，8位
域宽，只输出
4个字符，少
的部分前面补
空格。

注意：格式字符串中有转换说明时，其后数据项的数目、类型和顺序要和格式字符串中的转换说明一一匹配。

如果转换说明与输出参数不一致会怎么样？

- ◆ 如果**转换字符使用不当**或**转换说明的个数多于数据项个数**，则会输出错误的数据；
 - **关于输出类型不匹配可能引发的输出问题**，请参见：
<https://www.zhihu.com/question/29300881>
- ◆ 如果**转换说明的个数少于数据项个数**，则多出的数据项不会被输出。

例如：设 **int i=-6; double x=5.7, y=123.4567;**

(1) `printf("%d,%f\n", i, y, x );`

输出：-6,123.456700

数据项多了，因无对应的转换说明，x未被输出。

(2) `printf("%d\n", x );`

输出：-858993459

x的类型与转换说明%d不匹配。此时**将x截断**，取低32位、视为整数编码，按整数解析并输出。

(3) `printf("i=%d\n");`

输出：**i=4109**

参数里缺少数据项，随机输出一个不确定的值。

输出无符号数： **%d**用于输出有符号数、**%u**用于输出无符号数。

如：设 `int i = 100; unsigned j = 100;`

```
printf("%d,%u", i, i); //输出: 100,100
```

```
printf("%d,%u", j, j); //输出: 100,100
```

如：设 `int x = ~0; unsigned y = ~0;`

```
printf("%d,%u", x, x); //输出: -1,65535
```

```
printf("%d,%u", y, y); //输出: -1,65535
```

强调：输出时仅按照**转换说明符**来解释后面**数据项的字节内容**，而不是由数据项的类型决定输出内容。对于同样的字节内容，只要**类型相容**，就**按照转换说明的类型解释其中二进制位串的值并输出**。

关于printf的格式字符串，还有以下内容：

- ◆ 浮点数的其他输出形式：%e, %g
- ◆ 特殊转换字符n、%的用法
- ◆ **h, l, L的用法**
- ◆ 可变域宽
- ◆ 八进制、**十六进制数的输出、指针型数据的输出等**

请阅读课件后面的内容及教材的相关内容，自学。

转换字符（详见P70 表3-1），重点掌握c、d、f、s

转换字符	参数类型	输出格式
d , i	int	十进制整数
o	int	八进制整数（不带前缀0）
x , X	int	十六进制数（不带前缀0x或0X）
u	int	无符号十进制数
c	char	单个字符
s	char *	字符串（必须以' \0' 结束或在域宽说明中给出长度限制）
f	double	小数形式的浮点数（小数部分位数由精度确定，缺省为6位）
e, E	double	标准指数形式的浮点数（尾数部分位数由精度确定）
g, G	double	在不输出无效零的前提下，按输出域宽度较小的原则从%f和%e两种格式中自动选择
p	void *	指针值（输出格式与具体实现有关）
n	int *	将到达该转换说明为止已输出的字符数目写入对应的参数中，不转换参数
%	不转换参数	输出一个%字符



浮点数的其他输出形式：%e, %g

%f、%e、%g区别在于缺省域宽说明时的输出精度

◆**%f**：输出小数形式的浮点数，小数部分为6位。

多于6位采用四舍五入，少于6位则在末位补零，以保证6位小数；

◆**%e**：输出标准**指数形式**的浮点数，尾数部分为6个有效数字。

包括1位非零的整数部分和5位小数部分，采用四舍五入和末位补零的方法确保6个有效数字；

◆**%g**：将按%f和%e两种格式输出的数据去掉无效零后进行比较，
选取输出宽度较小的那种格式进行输出。

例：设变量说明为

```
double x=12.00004951, y=12.4951, z=12000000.4951;
```

则

```
printf("%f\t%e\t%g\n%f\t%e\t%g\n", x, x, x, y, y, y);
```

```
printf("%f\t%e\t%g\n", z, z, z);
```

输出：

12.000050	1.20000e+001	12
12.495100	1.24951e+001	12.4951
12000000.495100	1.20000e+007	1.2e+007

特殊转换字符n、 %的用法



自学

◆ 转换字符n的用法。

设有：int x = 1;, 依次执行下面两个语句：

```
printf( "abc%n" , &x);
```

```
printf( "x=%d" , x);
```

输出：abc

3

分析：第一个语句首先输出%n之前的字符abc，然后统计到%n为止已输出的字符数目（这里是3），最后该字符数目写到按对应参数所表示的内存地址中去（这里&x表示变量x的地址），所以x的值被改写为3；

第二个语句输出整数x的当前值，即3。

◆ 转换字符%的用法。

情况1: **printf("%%d", 100);**

输出1: **%d**

分析1: **相邻的两个%输出一个%**, 字符d作为普通字符输出。
而后面的数据项100不被输出。

情况2: **printf("%%%d" , 100);**

输出2: **%100**

分析2: 前两个相邻的%输出一个%, 再后面的%d是输出十进制整数的转换说明。

特别说明:



自学

- ◆ 如果%后面是一个%, 则输出一个%;
- ◆ 如果%与其后的字符不能构成一个合法的转换说明, 大多数编译系统将%与其后面的字符一起作为普通字符输出。

例如: 设 `int i=-6; double x=5.7, y=123.4567;`

(1) `printf("%d%%100", -i/2);`

输出: **3%100**

- 相邻的两个%输出一个%, 100原样输出。

(2) `printf("%a", i);`

输出: **%a**

- 因为%后面的a不是转换字符, %a被当作普通字符输出。
- 同时由于格式字符串中没有转换说明, 所以后面的数据项i没被输出。

h, l, L的用法： 分别用于输出short int、 long int/double、
long double类型的数据

自学

例3.10 设： short a; long b;
 double x; **long double y;**

printf("a=%**hd**,b=%**ld**,x=%**lf**,y=%**Lf**", a, b, x, y);

输出短整数

输出长整数

输出double类型的浮点数

输出long double类型的浮点数

可变域宽

自学

*****: **可变域宽**, 代表一个整数, 其值由对应的参数决定, 可用于替代m和n表示的域宽或精度, 可随参数变化。

例3.11 : 设 `int max=6; char s[10]="123456789";`

(1) `printf("%*c", max, ' ' );`

输出: 6个空格

*代表的域宽由对应参数max决定(6)

(2) `printf("%*s", 2*max, s);`

输出: 123456789

*代表的域宽由对应参数2*max决定(12)

左边填充了3个前导空格

(3) `printf("%.s", max, s);`

输出: 123456

*代表的输出精度由对应参数max决定(6)

2. 格式化输入函数scanf

函数原型: `int scanf (const char *format, .....);`

格式字符串 其它参数

功能：按照**格式字符串**说明的格式，接收从键盘输入的数据，这些数据将被依次保存到**其它参数**指示的存储位置中。返回值是实际输入的数据的个数。遇到文件尾或执行出错时返回**EOF**。

format: 格式字符串，与printf函数的格式字符串相似，也是由转换说明和普通字符两部分组成。

(2) scanf函数的格式字符串

scanf函数的格式字符串与printf函数相似，由**转换说明**和**普通字符**两部分组成：

(1) **转换说明**：以**%**开头，后跟**转换字符**

- 基本形式：**%转换字符**
- 常用的scanf函数转换字符如P63表3.3所示

(2) **普通字符**：指转换说明之外的其它字符

又分为**空白字符**和**非空白字符**。

scanf函数格式字符串中最常见的转换说明形式是：

%转换字符

- ◆ 一般不含域宽等内容。

重点掌握以下4个转换字符：

- ◆ **d**： **%d**， 输入整数
- ◆ **c**： **%c**， 输入字符
- ◆ **f**： **%f**， 输入浮点数
- ◆ **s**： **%s**， 输入字符串

另，关于普通字符的问题见后

scanf的调用

必须有



scanf (格式字符串, 输入参数1, 输入参数2, ..., 输入参数n);

输入参数列表

输入参数列表:

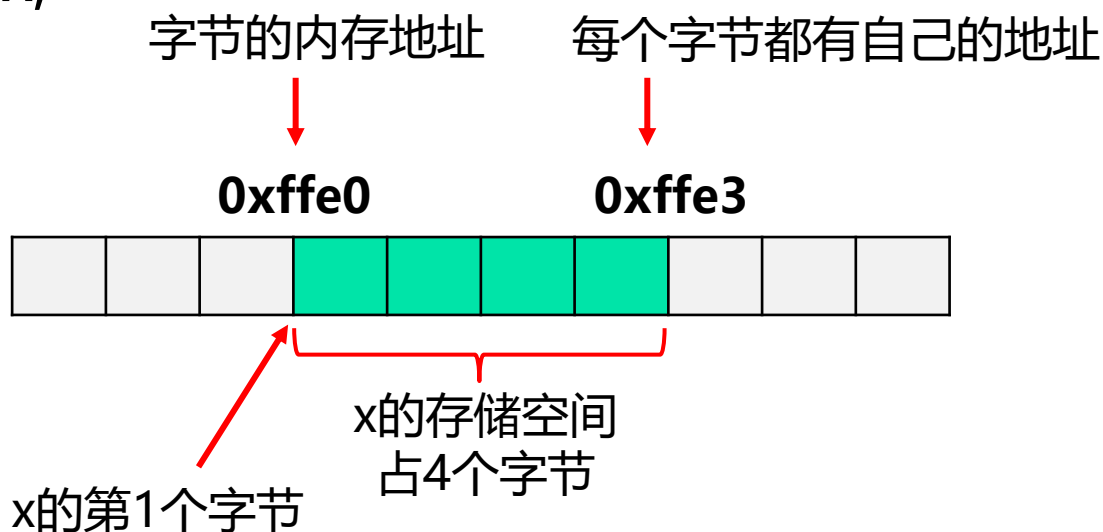
- ◆ 至少应有一个输入参数。
- ◆ 各输入参数在**类型**、**数目**和**顺序**上必须与格式字符串中的转换说明一致。
- ◆ **输入参数必须是地址型的量**, 可以是基本类型变量的地址或指针类型变量, 如取变量的地址: **&x** 或 **数组名**

地址型数据和变量的地址：

表示**内存地址**的数据称为**地址型数据** (详细见第8章的相关内容)。

变量的地址：变量所占存储空间中**第一个字节的地址**称为**变量的地址**，如下图中，变量x的地址是**0xffe0**。

设 int x;



x的第一个字节的地址称为x的地址，0xffe0

如何获取变量的地址?

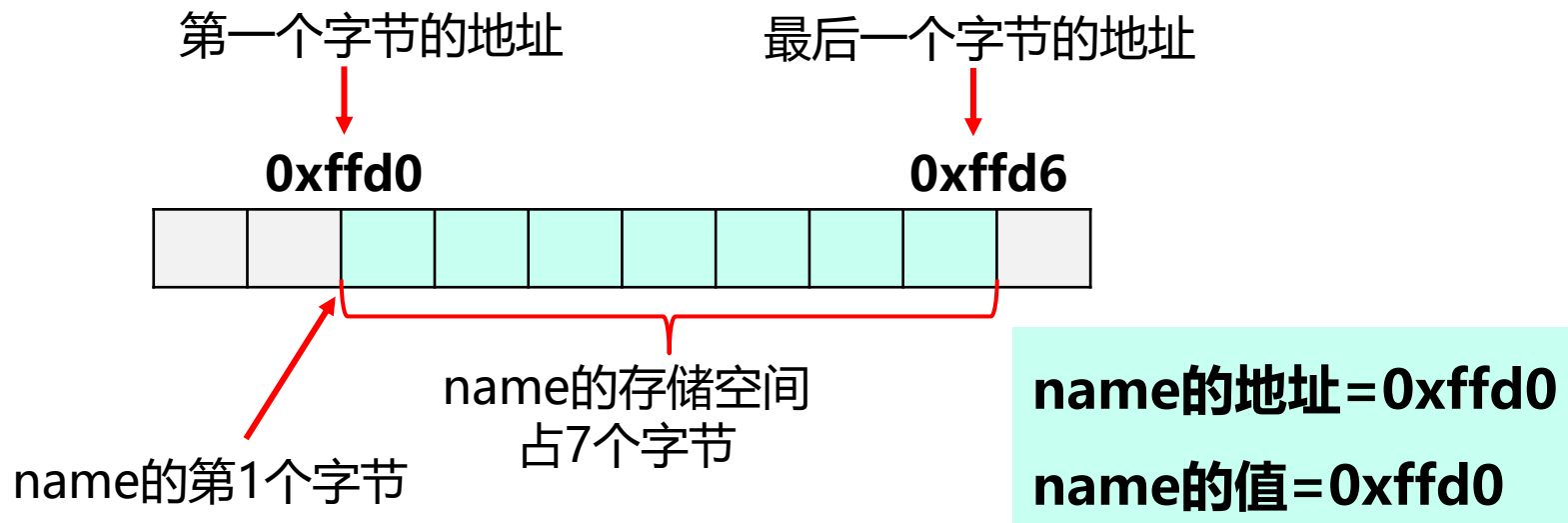
a) 取地址运算: **&**

如: **int x;** **&x** **char c;** **&c** **double d;** **&d**

b) **数组名**就代表数组地址 **禁止用**&**"取"数组的地址**

如: char name[7];

&name ✗



scanf使用举例

1) 输入简单变量

如: `int x; unsigned y; char c; float f;`

`scanf( "%d%u%c%f" , &x, &y, &c, &f );`

↑
格式字符串中包含
转换说明符

↑
输入参数表中列出接
收数据的变量的地址

2) 输入字符串

如: `char name[20];`

`scanf("%s", name);`

注: 数组名就代表数组地址,
禁止用&取数组名的地址。

`scanf("%s", &name);` ❌



scanf也是从**输入流**中读取字符，然后按照格式字符串中**转换说明规定的格式**将输入转换成相应类型的数据后，依次存放到由输入参数指定的内存单元中。

scanf 对输入形式的要求:

- (1) 格式字符串中, 转换说明之间可以不用空白符 (**空格、制表符**) 隔开, 但输入时一般用空白符将各输入项隔开。

如: `scanf("%d%s%f", &x, name, &f);`

输入: 123 wang 3.14

清晰

否则可能无法正确区分输入的内容, 如:

输入: 123 wang3.14

但是有例外：

- ◆ 如果**输入数字（整数或浮点数）**后紧接着**输入字符时**，**输入的数字和字符间不要加空白**，否则数字后面的空白符将被当作字符读入而出错。

如：scanf("%d%c", &x, &c);

输入：123a



数字和字符连在一起写

若输入：123 a



这个空格会被当作字符读入到c中，
而后面的a读不进去。

- ◆ 如果**输入字符后紧接着输入数字**，输入的字符和数字之间可以**不加空白**，scanf函数能正确区分字符和后面数字。

如：scanf("%c%d", &c, &x);

输入：a123



字符和数字可以连在一起写

若输入：a 123



中间加了空白符（即使是多个）也没问题，scanf可以自动过滤掉“非必要”的空白符，而把数字正确读入x中。

- ◆ 如果**输入数字（整数或浮点数）、字符后紧接着输入字符串**，**输入的数字或字符和后面的字符串之间可以不加空白**，scanf函数可以正确区分它们。

如：scanf("%d%s", &x, name);

输入：123wang ✓ 数字和字符序列可以连在一起写

若输入：123 wang ✓

中间加了空白符（即使是多个）也没问题，scanf可以自动过滤掉“非必要”的空白符，而把wang正确读入name中。

如：scanf("%c%s", &c, name);

输入：awang ✓

一个字符和后面的字符串可以连在一起写：
a作为独立字符读入c，后续的其它字符序列
处理成字符串后读入name。

若输入：a wang ✓

中间加了空白符（即使是多个）也没问题，
scanf可以自动过滤掉“非必要”的空白符，而
把wang正确读入name中。

- ◆ 但如果输入字符串后紧接着输入数字，字符串和数字之间必须加空白，否则scanf函数无法辨别数字是否是字符串的一部分。

如：scanf("%s%f", name, &f);

输入： wang 3.14 ✓

字符序列和数字要隔开

若输入： wang3.14 ✗

注意：输入的字符串内部不能有空白字符，否则scanf无法读入空白后面的内容。

例 设有：

```
char s[20];  
scanf("%s", s);
```

若输入： **Wang Duoqiang**
则s中只会读入**Wang**。

原因：空白字符是scanf解析输入的分隔符。 scanf在扫描输入时，碰到空白符就认为一个输入项结束。所以本例中读入Wang后，后面的空格表示当前一个连续的字符序列读完了（不包含空格），并且scanf只有一个输入，所以马上就返回，后面的Duoqiang没有被读入。



如果需要输入一个带空白字符的字符串，怎么办？

◆ 用gets函数来实现： `gets(name);`

gets函数仅以回车键（换行符'\n'）作为输入的结束，中间出现的空格不影响输入，故而可以输入一行文字，包括其中的空白符。

(2) 格式字符串中，**转换说明之间可以用空白符隔开**，此时对输入不产生任何影响（还是以上的一些要求）。

如：scanf("%c %d %s %f", &c, &x, name,&f);

转换说明之间加空白（可以多于一个），
对输入不产生影响

输入：a123wang 3.14 ✓

或输入：a 123wang 3.14 ✓

或输入：a 123 wang 3.14 ✓

(3) 格式字符串中如果存在**非空白的其它普通字符**，则要求输入时必须在对应位置**输入这些字符**。否则输入出错。

如：scanf("%d,%d", &x, &y);

↑
这里有一个逗号

输入：123,567 ✓

或：123,

567 ✓

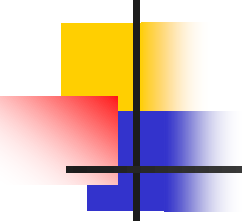
↖ ↗
必须在对应位置输入逗号

↖
这样的空白
还是可以加

若输入：123 567 ✗

少了逗号

自行思考：scanf("x=%d y=%d", &x, &y);



所以scanf的格式字符串中一般**不要写除空白符**

以外的其它普通字符，否则会造成输入的麻烦！

(4) 如果转换说明与输入参数之间的类型、个数不匹配，会导致执行异常，甚至程序非正常终止；

例如： 设 `int i, j;`

`scanf("%d%d", &i, &j);`

输入： `12a` ✓

结果： `i=12`， `j`未赋值， 函数返回1

分析： `%d`用于输入十进制整数。这里12作为整数读入`i`，但后面是字符`a`，不是合法的十进制数码，无法做第二个整数的转换，故而`j`未被赋值，而且此时因**出现执行异常**，`scanf`终止执行。由于实际正确读入的数据个数是1，所以`scanf`返回1.

再如：

◆ `scanf("%d%d", &i);`

分析：这里有两个转换说明，但只有一个输入参数，输入的第二个数据没法合法接收，此种情况可能会造成系统崩溃而死机（**随机存储，内存溢出**）。

◆ `double x;`

`scanf("%f", &x);`

`printf("%f", x);`

输入：3.14159

输出：0.00000

分析：%f用于输入float型浮点数，而x是double型，两者类型不匹配，scanf无法完成正确输入。

注：输入double数据用 `scanf("%lf", &x)`。



scanf函数的其它转换字符

◆ **n**: 统计前面成功处理过的字符数

例, 设有: int i, j, k;

```
scanf("%d%d%n", &i, &j, &k)
```

输入: 10 20 30 ✓

结果: 10被赋给i, 20被赋给j, 6赋给了k。

说明: 对应第三个转换说明%n, 不是从输入流中读取数据30, 而是把**到此为止已读入的字符数**赋给k (包括%n自身和之前的空格字符), 故k等于6。30没有被读入。

◆ %:

- (1) 如果转换说明为**%%**，则不从输入流读取数据，对应参数的值不改变；

例如：设有 `int i=1`；

```
scanf("%%d", &i);
```

```
printf("i=%d", i);
```

输入: **100** ✓ (或**%100** ✓)

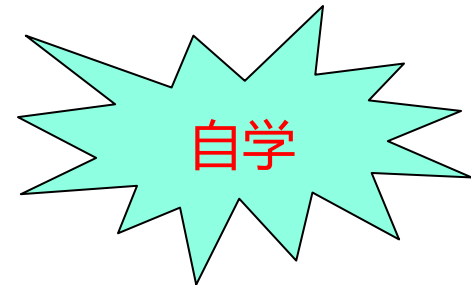
输出: **i=1**

- (2) 如果在其他完整的转换说明前面有两个相邻的%，则输入时需要一个%与之匹配。

如：上例改成`scanf("%%%d", &i);` 则要输入: **%100**



scanf函数转换说明的一般形式:



%[选项]转换字符

选项包括哪些?

- ◆ **域宽**: 从**自然输入域**首字符开始取指定宽度的字符作为**实际输入域**。
- ◆ **h, l和L**: 转换修饰字符, 用于输入**短整数(%hd)**、**长整数(%ld)**、**双精度浮点数(%lf)**、**长双精度浮点数(%Lf)**
- ◆ **虚读**: 在转换字符前面使用 **"*"** (**%*d**) , **赋值抑制符**

常用的scanf函数转换字符如P74表3.2所示

a) **域宽**：如果指定了域宽，则从自然输入域首字符开始取指定宽度的字符作为实际输入域。



例 3.22 设有 `int i, j;`

```
scanf("%3d %d", &i, &j);
```

输入：1234 5678

结果：i=123

j=4

5678保留在输入流中

- 若自然输入域的宽度小于等于指定的域宽，则整个自然输入域将作为实际输入域；
- 若自然输入域的宽度大于指定域宽，则从自然输入域取完指定宽度的字符后，剩下的字符作为下一个自然输入域。

b) **h**, **l**和**L**: 输入短整数(**%hd**)、长整数(**%ld**)、双精度浮点数(**%lf**)、长双精度浮点数(**%Lf**)。

例 3.24 设有

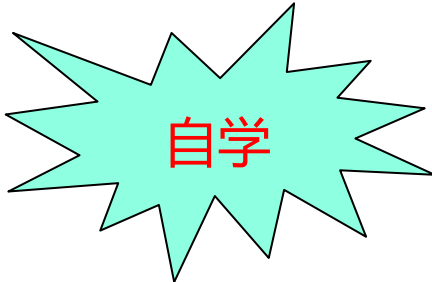
short i;

long j;

double x;

long double y;

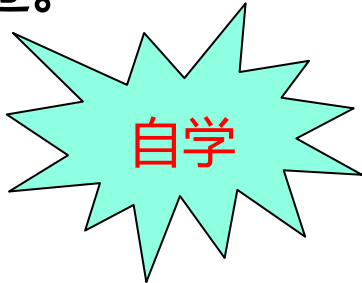
scanf("%hd%ld%lf%Lf", &i, &j, &x, &y);



自学

c) **抑制赋值符'*********'**：如果在某个转换字符前面使用了 **"*"**，则与该转换说明对应的输入域被跳过。

——这种情况称为 **"虚读"**。



自学

虚读用于从输入流中有选择地读取部分内容。

例3.25： 设有 `int n1,n2,n3;`

```
scanf( "%d %*d %d" ,&n1,&n2,&n3);
```

输入： **11 22 33 44**

结果： n1、 n2、 n3的值分别是： **11 33 44**



22被虚读，导致33被赋给n2

◆ 诸如printf和scanf，它们的**参数个数是可变的**。

——称之为**可变形参**，定义形式：

```
int printf(const char *format, ...);
```

```
int scanf (const char *format, ...);
```



其中，**“,...”** 是**可变参数**的表示形式，表示该函数在调用时除前面的字符串参数必需之外，其余参数的数目可变，可以是0个到多个。

具体参见函数一章的内容。

本章小结

本章介绍了C语言常用的标准输入与输出函数，包括

- 单个字符输入输出函数 `getchar`与 `putchar`
- 字符串输入输出函数 `gets` 和 `puts`,
- 格式化输入输出函数 `scanf` 和 `printf`
- ◆ 格式化输入输出中常用转换说明 `%c`、`%d`、`%s`、`%f`的用法需要记住。



课后作业

C语言的输入输出非常灵活，要在编程实践中不断探索，积累经验，熟练掌握它们的用法。

阅读第三章，将其中的例题（有些课堂上没有讲）和课件中的例题自己编程做一遍，学习C语言的这些标准输入输出函数的应用。